

Quelques algorithmes de tri

BUT Informatique - S5 - Qualité algorithmique

1 Recherche linéaire dans un tableau

Question 1

Écrivez l'algorithme de **recherche linéaire** qui, pour un entier e et un tableau tab donnés en entrée, renvoie la **première** position i du tableau tab pour lequel $tab[i] = e$ si e est dans tab , et renvoie -1 sinon.

solution

```
public static int recherche_lineaire(Integer e, Integer[] tab){
    int i = 0;
    while (i < tab.length && e != tab[i])
        i++;
    return (i < tab.length) ? i : -1;
}
```

Question 2

Quel invariant (pour la boucle `while`) pourrait-on utiliser pour démontrer que cet algorithme est correct ?

solution

On considère l'invariant qui spécifie qu'à chaque itération i de la boucle `while`, l'entier recherché n'apparaît dans aucune position strictement inférieure à i . Plus formellement:

$$Inv(i) = e \notin tab[0, \dots, i - 1]$$

On peut "démontrer" la correction à l'aide de quelques commentaires dans le code:

```
public static int recherche_lineaire(Integer e, Integer[] tab){
    int i = 0;
    // e n'est pas dans tab[0,...,-1] car ce tableau est vide
    while (i < tab.length && e != tab[i])
        // soit k la valeur de i à l'itération courante, supposons que e n'est pas dans tab[0,...,
        i++;
        // e n'est pas non plus dans tab[0,...,k] car e != tab[k]
    // par conservation, e n'est pas dans tab[0,...,i-1] et le while a terminé car
    // - soit i = tab.length et donc e n'est pas dans le tableau
    // - soit i < tab.length et e = tab[i], et donc i est la *première* position où e apparaît
    return (i < tab.length) ? i : -1;
}
```

Plus précisément, il faut démontrer les 3 propriétés pour cet invariant.

- **initialisation** $Inv(0)$: évidemment satisfaite car $tab[0, \dots, i-1]$ désigne une portion de tableau vide.
- **conservation**: au début de l'itération i du corps de la boucle (c'est-à-dire lorsque la ligne 3 a été exécutée mais pas encore la ligne 4), on fait l'hypothèse que $Inv(i)$ est satisfait, et on montre qu'à la fin de l'itération (c'est-à-dire après exécution de la ligne 4) $Inv(i')$, où i' est la valeur de la variable i à cet étape de l'algorithme, est encore satisfait. On a donc $e \notin tab[0, \dots, i-1]$ et, d'après la condition du `while` qui est vérifiée, i satisfait la condition $e \neq tab[i]$. Par conséquent, $e \notin tab[0, \dots, i]$, et comme $i' = i + 1$ (d'après l'incréméntation réalisée à la ligne 4) on a $e \notin tab[0, \dots, i' - 1]$, ce qui montre que $Inv(i')$ est satisfait.
- **terminaison** : à la sortie de boucle (i.e. après la ligne 5), i a une valeur qui satisfait soit $i = n$ (où est la longueur du tableau) soit $e = tab[i]$. Dans le premier cas, la fonction renvoie -1 et comme, par conservation, $Inv(i-1)$ est satisfait, on a $e \notin tab[0, \dots, n-1]$. Donc la fonction renvoie bien -1 quand l'entier recherché n'est pas présent dans le tableau. Dans le 2ème cas, $Inv(i-1)$ et $e = tab[i]$, donc e est trouvé en position i , n'apparaît pas avant dans le tableau et la fonction renvoie i . Conclusion: l'algorithme est correct. $\square$

Question 3

Déterminer, par le calcul, la coût $T(n)$ de cet algorithme dans le pire cas et dans le meilleur cas.

solution

```
1 public static int recherche_lineaire(Integer e, Integer[] tab){
2     int i = 0; // c1
3     while (i < tab.length && e != tab[i]) // c2 * (w+1)
4         i++; // c3 * w
5     return (i < tab.length) ? i : -1; // c4
6 }
```

Le coût de cet algorithme est

$$T(n) = c_1 + c_2(w + 1) + c_3w + c_4$$

où c_1 est le temps (supposé constant) d'exécution de l'affectation $i=0$, c_2 celui de la condition du `while` etc. w est le nombre de fois que l'itération `while` sera réalisée et appartient à l'intervalle $[0, \dots, n]$. Ce nombre dépend des entrées et pas seulement de la longueur du tableau. Dans le meilleur cas, $w = 0$ (c'est celui où e est trouvé en première position), et donc $T(n) = c_1 + c_2 + c_3$ qui est une fonction constante, d'où $T(n) \in \Omega(1)$. Dans le pire cas, $w = n$, c'est à dire $T(n) = (c_2 + c_3)n + c_1 + c_2 + c_4$ qui est une fonction linéaire, d'où $T(n) \in O(n)$.

2 Recherche dichotomique

Question 1

Écrivez l'algorithme *itératif* de **recherche dichotomique** qui optimise la recherche d'un élément dans un tableau **trié** en commençant par comparer l'élément recherché avec celui du milieu du tableau et poursuit la recherche dans la première ou la deuxième moitié du tableau en fonction du résultat de la comparaison.

solution

```
public static int recherche_dichotomique(Integer e, Integer[] tab){
    /* hypothèse: tab est trié dans l'ordre croissant */
}
```

```

int deb = 0;
int fin = tab.length-1;
int milieu = (fin+deb)/2;
while (deb!=fin){
    if (e>tab[milieu])
        deb = milieu+1;
    else fin = milieu;
    milieu = (fin+deb)/2;
}
return (tab[deb].equals(e)) ? deb : -1;
}

```

Question 2

Calculez le coût de cet algorithme dans le pire cas et dans le meilleur cas.

solution

```

1 public static int recherche_dichotomique(Integer e, Integer[] tab){
2     /* hypothèse: tab est trié dans l'ordre croissant */
3     int deb = 0; // c1
4     int fin = tab.length-1; // c2
5     int milieu = (fin+deb)/2; // c3
6     while (deb!=fin){ // c4*(w+1)
7         if (e>tab[milieu]) // c5*w
8             deb = milieu+1; // c6*w'
9         else fin = milieu; // c7*w''
10        milieu = (fin+deb)/2; // c8*w
11    }
12    return (tab[deb].equals(e)) ? deb : -1; // c_9
13 }

```

w représente le nombre de fois qu'est réalisée la boucle `while` et $w = w' + w''$. La difficulté dans le calcul rigoureux du coût est le coût de la conditionnelle `if`. On peut cependant encadrer ce coût. Ainsi:

$$\begin{aligned}
 T(n) &\leq c_1 + c_2 + c_3 + (w+1)c_4 + w(c_5 + \max(c_6, c_7) + c_8) + c_9 \\
 &\leq (c_4 + c_5 + \max(c_6, c_7) + c_8)w + c_1 + c_2 + c_3 + c_4 + c_9 \\
 &\leq aw + b
 \end{aligned}$$

De même,

$$\begin{aligned}
 T(n) &\geq (c_4 + c_5 + \min(c_6, c_7) + c_8)w + c_1 + c_2 + c_3 + c_4 + c_9 \\
 &\geq a'w + b
 \end{aligned}$$

À chaque itération, l'intervalle $fin - deb$, initialement égal à n , est divisé par 2 (en division entière). La boucle termine quand $fin - deb = 0$. Par conséquent, $w = \log_2(n)$ et donc le coût $T(n)$ est une fonction logarithmique de n dans le pire cas comme dans le meilleur, c'est-à-dire $T(n) \in \Theta(\log(n))$.

Si dans l'algorithme on sort précocément de la boucle `while` dès que $e == tab[milieu]$, alors dans le meilleur cas e est trouvé dès la première itération, c'est-à-dire $w = 0$, et le coût est constant $T(n) \in \Omega(1)$. Dans le pire cas, il n'est pas trouvé (ou trouvé à la dernière itération) et le coût est logarithmique: $T(n) \in \Theta(\log(n))$.

3 Tri à bulles

Le tri à bulles consiste à successivement comparer les cases adjacentes d'un tableau de valeurs et à les échanger de manière à faire remonter les plus grandes valeurs vers le bout du tableau. Ainsi, à la fin de la première itération, la plus grande valeur du tableau se trouve en dernière position. À la fin de la deuxième itération, la deuxième plus grande valeur se trouve en avant-dernière position, etc.

Question 1

Écrivez la méthode

```
static void swap(int [] tab, int idx, int idx2)
```

qui permet d'échanger deux valeurs d'un tableau d'entiers. Ensuite, écrivez cette fonction pour écrire la méthode

```
static void bubbleSort(int [] tab)
```

qui permet de réaliser le tri à bulles. Dans cette première version, vous utiliserez deux boucles for.

solution

```
static void swap(int [] tab, int idx, int idx2){
    int tmp = tab[idx];
    tab[idx] = tab[idx2];
    tab[idx2] = tmp;
}

static void bubbleSort(int [] tab){
    for(int i=tab.length-1; i>0; i--){
        for(int j=0; j<i; j++){
            if(tab[j]>tab[j+1]){
                swap(tab,j,j+1);
            }
        }
    }
}
```

Question 2

Calculer le coût $T_1(n)$ de cet algorithme (n désigne le nombre d'éléments du tableau à trier).

solution

```
1 static void bubbleSort(int [] tab){
2     for(int i=tab.length-1; i>0; i--){ // c2 * n
3         for(int j=0; j<i; j++){ // c3 * (n + n-1 + ... + 2)
4             if(tab[j]>tab[j+1]){ // c4 * (n-1 + n-2 + ... + 1)
5                 swap(tab,j,j+1); // c5 * ?
6             }
7         }
8     }
9 }
```

Le coût de la ligne 5 dépend du résultat de la comparaison en ligne 4:

- dans le pire cas, la comparaison est toujours satisfaite (ie. les valeurs du tableau sont initialement triées dans l'ordre décroissant) et la méthode swap est appelée autant de fois que la ligne 4, donc le coût est $c_5(n - 1 + n - 2 + \dots + 1)$
- dans le meilleur cas, la comparaison n'est jamais satisfaite (ie. les valeurs du tableau sont initialement triées dans l'ordre croissant) et la méthode swap n'est jamais appelée, donc le coût est nul.

Donc, dans le pire cas, le coût de l'algorithme est:

$$T_1(n) = c_2n + c_3 \sum_{i=2}^n i + c_4 \sum_{i=1}^{n-1} i + c_5 \sum_{i=1}^{n-1} i = \frac{c_3 + c_4 + c_5}{2}n^2 + (c_2 + \frac{c_3 - c_4 - c_5}{2})n - c_3$$

et dans le meilleur cas:

$$T_1(n) = c_2n + c_3 \sum_{i=2}^n i + c_4 \sum_{i=1}^{n-1} i = \frac{c_3 + c_4}{2}n^2 + (c_2 + \frac{c_3 - c_4}{2})n - c_3$$

Dans les deux cas, le coût est de la forme $T_1(n) = an^2 + bn + c$ pour certaines constantes a , b et c , c'est-à-dire $T_1 \in \Theta(n^2)$.

Question 3

Appliquez le tri à bulle au tableau suivant:

22	11	8	17	25	34	47	52	69	98
0	1	2	3	4	5	6	7	8	9

Que remarquez-vous?

solution

On constate qu'il y a peu de valeurs à trier et que l'on continue à comparer des valeurs alors que le tableau est déjà trié.

Question 4

Pour pallier à cela, modifiez le code de la question 1 afin d'éviter tout parcours supplémentaire inutile.

solution

On remarque dans l'algorithme bubbleSort que l'invariant "`tab[i, ..., tab.length-1]` est trié" est satisfait au début (et à la fin) de chaque itération sur `i`. On remarque aussi qu'à la fin de chaque itération sur `i`, si on note `perm` la dernière valeur de l'indice `j` (de l'intervalle `[0, i]`) qui a réalisé une permutation entre les éléments d'indices `j` et `j+1`, alors `tab[perm, ..., i]` est nécessairement déjà trié (sinon il y aurait eu une permutation dans cet intervalle). Par conséquent, `tab[perm, ..., tab.length-1]` est trié et il suffit de reprendre le tri à bulle sur l'intervalle `tab[0, ..., perm]` plutôt que sur `tab[0, ..., i-1]` comme dans bubbleSort.

```

1 static void bubbleSortOpt(int [] tab){
2     int end = tab.length - 1;
3     while(end > 0){
4         int perm = -1;
5         for(int i=0; i<end; i++){

```

```

6         if(tab[i]>tab[i+1]){
7             swap(tab,i,i+1);
8             perm = i;
9         }
10    }
11    end = perm;
12 }
13 }

```

Question 5

Calculer le coût de cet algorithme optimisé dans le pire cas et dans le meilleur cas.

solution

```

1 static void bubbleSortOpt(int [] tab){
2     int end = tab.length - 1;          // c1
3     while(end > 0){                    // c2 * (w+1)
4         int perm = -1;                 // c3 * w
5         for(int i=0; i<end; i++){      // c4 * ((e_1+1) + ... + (e_w+1))
6             if(tab[i]>tab[i+1]){        // c5 * (e_1 + ... + e_w)
7                 swap(tab,i,i+1);       // c6 * (e_1' + ... + e_w')
8                 perm = i;              // c7 * (e_1' + ... + e_w')
9             }
10        }
11    end = perm;                         // c8 * w
12 }
13 }

```

w est le nombre de fois qu'est exécuté *le corps* de la boucle `while` et qui dépend du tableau. $e_1, \dots, e_w$ sont les valeurs successives de la variable `end` (donc, e_1 vaut $n-1$ car `end` est initialisée avec `tab.length-1`). Enfin, e'_j est le nombre de fois que la condition `tab[i]>tab[i+1]` est satisfaite dans la j ème itération de la boucle `while` (e'_j vaut au plus e_j). On en déduit:

$$\begin{aligned}
 T_2(n) &= c_1 + c_2(w+1) + c_3w + c_4\left(w + \sum_{j=1}^w e_j\right) + c_5 \sum_{j=1}^w e_j + (c_6 + c_7) \sum_{j=1}^w e'_j + c_8w \\
 &= (c_1 + c_2) + (c_2 + c_3 + c_4 + c_8)w + (c_4 + c_5) \sum_{j=1}^w e_j + (c_6 + c_7) \sum_{j=1}^w e'_j
 \end{aligned}$$

Le pire cas correspond à celui où le tableau est trié par ordre décroissant: la boucle `while` est réalisée $w = n-1$ fois (comme dans la version originale du tri à bulles), les valeurs successives de `end` sont $e_1 = n-1, e_2 = n-2, \dots, e_{n-1} = 1$ et celles de e'_j sont maximales, ie. $e'_1 = e_1 = n-1, e'_2 = e_2 = n-2, \dots, e'_{n-1} = e_{n-1} = 1$. D'où, $\sum_{j=1}^w e_j = \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2} = \sum_{j=1}^w e'_j$. On en déduit que $T_2(n)$ est une *fonction quadratique* de n , d'où $T(n) = O(n^2)$.

Le meilleur cas correspond à celui où le tableau est trié par ordre croissant: la boucle `while` n'est réalisée qu'une seule fois $w = 1$, $e_1 = n-1$, et aucune permutation n'est réalisée: $e'_1 = 0$. D'où, $\sum_{j=1}^1 e_j = n-1$ et $\sum_{j=1}^1 e'_j = 0$. On en déduit que $T(n)$ est une *fonction linéaire*, d'où $T(n) = \Omega(n)$.

4 Fusion de deux tableaux triés

Question 1

Écrivez un algorithme qui fait la fusion de deux tableaux d'entiers triés, dont le prototype serait le suivant:

```
void fuseTab(int[] tabInA, int[] tabInB, int[] tabOut)
```

Le tableau tabOut (préalablement alloué avec une taille suffisante avant l'appel de la fonction fuseTab) doit être trié également et l'algorithme doit faire un seul balayage des tableaux.

solution

```
void fuseTab(int[] tabInA, int[] tabInB, int[] tabOut){  
  
    int aIdx = 0, bIdx = 0, outIdx = 0;  
    while(aIdx < tabInA.length && bIdx < tabInB.length){  
        if(tabInA[aIdx] < tabInB[bIdx]){  
            tabOut[outIdx++] = tabInA[aIdx++];  
        }else{  
            tabOut[outIdx++] = tabInB[bIdx++];  
        }  
    }  
    while(aIdx < tabInA.length){  
        tabOut[outIdx++] = tabInA[aIdx++];  
    }  
    while(bIdx < tabInB.length){  
        tabOut[outIdx++] = tabInB[bIdx++];  
    }  
}
```

Question 2

Calculez le coût de l'algorithme.

solution

Il est clair que le coût est $n_a + n_b$ où n_a et n_b sont les longueurs des tableaux tabInA et tabInB.